Relational Calculus

CSE 341
Introduction to Database Systems
Winter 2024
© Royal Military College of Canada

Introduction

- Relational algebra and relational calculus are formal languages associated with the relational model.

- Informally, relational algebra is a (high-level) procedural language and relational calculus a non-procedural language.

- However, formally both are equivalent to one another.

- A language that produces a relation that can be derived using relational calculus is <u>relationally complete</u>.

Relational Calculus

- Relational calculus query specifies *what* is to be retrieved rather than *how* to retrieve it.
  - No description of how to evaluate a query.
- In first-order logic (or predicate calculus), *predicate* is a truth-valued function with arguments.
- When we substitute values for the arguments, function yields an expression, called a *proposition*, which can be either true or false.

Relational Calculus

- If the predicate contains a variable (e.g. '*x* is a member of staff'), there must be a range for *x*.

- When we substitute some values of this range for *x*, proposition may be true; for other values, it may be false.

- When applied to databases, relational calculus has two forms:
  - *Tuple* Relational Calculus (TRC) and
  - *Domain* Relational Calculus (DRC).

Tuple Relational Calculus (TRC)

- Interested in finding tuples for which a predicate is true. Based on use of tuple variables.
- Tuple variable is a variable that 'ranges over' a named relation, which means it takes on tuples from that relation as its values.
- When we specify that the *tuple variable S* ranges over the Staff relation, we write it as Staff(S).
  - This is stating that S is a tuple from the Staff table.

Tuple Relational Calculus

- Predicate P(S): This represents a condition or a set of conditions that tuples must satisfy. It is a logical statement that can include comparisons involving attributes of the tuple variable S.
- Finding Tuples with {S | P(S)}: This notation is used to describe a set of all tuples S for which the predicate P(S) is true. Essentially, it is a query that selects all the rows in the Staff relation, where the condition P(S) holds.

Tuple Relational Calculus - Example

- To find details of all staff earning more than £10,000:

  $\{S \mid Staff(S) \wedge S.salary > 10000\}$

- To find a particular attribute, such as salary, write:

  $\{S.salary \mid Staff(S) \wedge S.salary > 10000\}$

Tuple Relational Calculus

- Can use two *quantifiers* to tell how many instances the predicate applies to:
  - Existential quantifier $\exists$ ('there exists')
  - Universal quantifier $\forall$ ('for all')

- Tuple variables qualified by $\forall$ or $\exists$ are called *bound* variables, otherwise called *free* variables.

Tuple Relational Calculus

- Existential quantifier used in formulae that must be true for at least one instance, such as:

$$\textbf{Staff(S)} \wedge (\exists \textbf{B})(\textbf{Branch(B)} \wedge (\textbf{B.branchNo} = \textbf{S.branchNo}) \wedge \textbf{B.city} = \textbf{'London'}))$$

- Means 'There exists a Branch tuple with the same branchNo as the branchNo of the current Staff tuple, *S*, and is located in London'.

Tuple Relational Calculus

- Universal quantifier is used in statements about every instance, such as:

  $(\forall B)\ (B.city \neq \text{‘Paris’})$

- Means ‘For all Branch tuples, the address is not in Paris’.

- Can also use $\sim(\exists B)\ (B.city = \text{‘Paris’})$ which means ‘There are no branches with an address in Paris’.

Tuple Relational Calculus

- Formulae should be unambiguous and make sense.
- A (well-formed) formula is made out of <u>atoms</u>:
  - $R(S_i)$, **where** S_i **is a tuple variable and** R **is a relation**
  - $S_i.a_1 \ \theta \ S_j.a_2$
  - $S_i.a_1 \ \theta \ c$
- Can recursively build up formulae from atoms:
  - An atom is a formula
  - If F_1 and F_2 are formulae, so are their conjunction, $F_1 \wedge F_2$; disjunction, $F_1 \vee F_2$; and negation, $\sim F_1$
  - If F is a formula with free variable X, then $(\exists X)(F)$ and $(\forall X)(F)$ are also formulae.

Example - Tuple Relational Calculus

- List the names of all managers who earn more than £25,000.

$$\{S.fName, S.lName \mid Staff(S) \wedge S.position = \text{‘Manager’} \wedge S.salary > 25000\}$$

- List the staff who manage properties for rent in Glasgow.

$$\{S \mid Staff(S) \wedge (\exists P)\ (PropertyForRent(P) \wedge (P.staffNo = S.staffNo) \wedge P.city = \text{‘Glasgow’})\}$$

Example - Tuple Relational Calculus

- List the names of staff who currently do not manage any properties.

$$\{S.fName, S.lName \mid Staff(S) \wedge (\sim(\exists P)\ (PropertyForRent(P) \wedge (S.staffNo = P.staffNo)))\}$$

Or

$$\{S.fName, S.lName \mid Staff(S) \wedge ((\forall P)\ (\sim PropertyForRent(P) \vee \sim(S.staffNo = P.staffNo)))\}$$

Example - Tuple Relational Calculus

- List the names of clients who have viewed a property for rent in Glasgow.

$$\{C.fName, C.lName \mid Client(C) \wedge ((\exists V)(\exists P)$$
$$(Viewing(V) \wedge PropertyForRent(P) \wedge$$
$$(C.clientNo = V.clientNo) \wedge$$
$$(V.propertyNo=P.propertyNo) \wedge$$
$$P.city = \text{‘Glasgow’}))\}$$

Tuple Relational Calculus

- Expressions can generate an infinite set. For example:
  $\{S \mid \sim Staff(S)\}$

- To avoid this, add restriction that all values in result must be values in the domain of the expression.

Domain Relational Calculus (DRC)

- Uses variables that take values from <u>domains</u> instead of tuples of relations.

- If $F(d_1, d_2, \ldots, d_n)$ stands for a formula composed of atoms and $d_1, d_2, \ldots, d_n$ represent domain variables, then:

$$\{d_1, d_2, \ldots, d_n \mid F(d_1, d_2, \ldots, d_n)\}$$

is a general domain relational calculus expression.

Example - Domain Relational Calculus

- Find the names of all managers who earn more than £25,000.

$$\{fN, lN \mid (\exists\ sN, posn, sex, DOB, sal, bN)$$
$$(Staff\ (sN, fN, lN, posn, sex, DOB, sal, bN) \wedge$$
$$posn = \text{‘Manager’} \wedge sal > 25000)\}$$

"Retrieve the first and last names (fN, lN) of individuals for which there exist attributes (sN, posn, sex, DOB, sal, bN) in the Staff relation such that the individual's position is 'Manager' and their salary is greater than £25,000."

Example - Domain Relational Calculus

- List the staff who manage properties for rent in Glasgow.

$$\{sN, fN, lN, posn, sex, DOB, sal, bN \mid$$
$$(\exists\ sN1, cty)(Staff(sN, fN, lN, posn, sex, DOB, sal, bN) \wedge$$
$$PropertyForRent(pN, st, cty, pc, typ, rms,$$
$$rnt, oN, sN1, bN1) \wedge$$
$$(sN = sN1) \wedge cty = \text{‘Glasgow’})\}$$

"Retrieve the full set of attributes for staff members (sN, fN, lN, posn, sex, DOB, sal, bN) for whom there exists a matching staff number (sN1) and city (cty) in the **PropertyForRent** relation, where the staff member is associated with a property in Glasgow."

Example - Domain Relational Calculus

- List the names of staff who currently do not manage any properties for rent.

$$\{fN, lN \mid (\exists sN)$$
$$(Staff(sN,fN,lN,posn,sex,DOB,sal,bN) \wedge$$
$$(\sim(\exists sN1)\ (PropertyForRent(pN, st, cty, pc, typ,$$
$$rms, rnt, oN, sN1, bN1) \wedge (sN=sN1))))\}$$

Example - Domain Relational Calculus

- List the names of clients who have viewed a property for rent in Glasgow.

$$\{fN, lN \mid (\exists cN, cN1, pN, pN1, cty)$$
$$(Client(cN, fN, lN, tel, pT, mR) \wedge$$
$$Viewing(cN1, pN1, dt, cmt) \wedge$$
$$PropertyForRent(pN, st, cty, pc, typ,$$
$$rms, rnt, oN, sN, bN) \wedge$$
$$(cN = cN1) \wedge (pN = pN1) \wedge cty = \text{‘Glasgow’})\}$$

Domain Relational Calculus

- When restricted to safe expressions, DRC is equivalent to TRC restricted to safe expressions, which is equivalent to relational algebra.

- Means every relational algebra expression has an equivalent relational calculus expression, and vice versa.

Other Languages

- *Transform-oriented languages* are non-procedural languages that use relations to transform input data into required outputs (e.g. SQL).

- *Graphical languages* provide user with picture of the structure of the relation. User fills in example of what is wanted and system returns required data in that format (e.g. QBE – Query By Example).

Other Languages

- *4GLs* can create complete customized application using limited set of commands in a user-friendly, often menu-driven environment.

- Some systems accept a form of *natural language*, sometimes called a *5GL*, although this development is still at an early stage.

Next…

SQL